

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА



Отчёт по лабораторной работе №5

«Разработка одностраничного веб-приложения (SPA) с
использованием фреймворка Vue.JS»

по дисциплине «Web-разработка: Frontend»

Семестр 4

Выполнил: студент
Мавров Артём Николаевич
гр. J3213
ИСУ 466574

Отчёт сдан: 15.05.2026

Проверил: Добряков Д. И.

Санкт-Петербург
2026

Содержание

1	Задача	3
2	Ход работы	3
2.1	Исходная структура проекта	3
2.2	Реализация composables	4
2.2.1	useAsyncAction	4
2.2.2	useModal	4
2.2.3	useListFilter	5
2.3	Переход на Composition API (<script setup>)	5
2.4	Рефакторинг компонентов	7
2.5	Роутер и navigation guard	8
2.6	Работа с внешним API через axios	8
2.7	Запуск проекта	8
3	Вывод	8

Задача

В рамках данной домашней работы необходимо мигрировать ранее разработанное приложение платформы управления ML-пайплайнами MLPipe на фреймворк Vue.js с соблюдением следующих требований: подключение Vue Router для клиентской навигации; реализация работы с внешним API посредством axios; разумное деление на компоненты, демонстрирующее понимание компонентного подхода; использование composable-функций для выделения повторяющегося функционала в отдельные файлы.

Ход работы

Исходная структура проекта

За основу взят проект, разработанный в предыдущей домашней работе. Стартовая структура уже содержала API-слой, Pinia-хранилища и Vue Router, однако полностью отсутствовал слой composables, а все компоненты были написаны в стиле Options API. Задача состояла в рефакторинге с переходом на Composition API и выделении повторяющегося функционала.

Итоговая структура директории `src/`:

```
1 src/
2   api/
3     instance.js           # axios -
4
5     auth.js               # AuthApi
6     experiments.js        # ExperimentsApi
7     models.js             # ModelsApi
8     artifacts.js          # ArtifactsApi
9     index.js              # IoC -
10  composables/
11    useAsyncAction.js      # loading/error
12    async -
13    useModal.js            #
14
15    useListFilter.js       #
16
17  stores/
18    auth.js / experiments.js / models.js / artifacts.js / index.js
19  views/
20    LoginPage.vue / RegisterPage.vue / DashboardPage.vue
21    ExperimentsPage.vue / ExperimentDetailPage.vue
22    ModelsPage.vue / ArtifactsPage.vue / ProfilePage.vue
23  components/
24    AppSidebar.vue / AppTopbar.vue / SvgSprite.vue
25  layouts/
26    AppShell.vue
27  router/
28    index.js
29  main.js
```

Листинг 1: Структура проекта после миграции

Реализация composables

Главное нововведение данной работы — директория `src/composables/` с тремя composable-функциями, покрывающими весь повторяющийся функционал приложения.

2.2.1 useAsyncAction

Composable инкапсулирует паттерн `loading / error / execute`, который до рефакторинга дублировался на каждой странице с формами:

```
1 import { ref } from 'vue'
2
3 export function useAsyncAction() {
4   const loading = ref(false)
5   const error   = ref('')
6
7   async function execute(fn) {
8     error.value   = ''
9     loading.value = true
10    try {
11      return await fn()
12    } catch (e) {
13      error.value =
14        e.response?.data?.error || e.message || '
15
16      throw e
17    } finally {
18      loading.value = false
19    }
20  }
21
22  function clearError() { error.value = '' }
23
24  return { loading, error, execute, clearError }
```

Листинг 2: Файл `src/composables/useAsyncAction.js`

Composable применяется в: `LoginPage`, `RegisterPage`, `ExperimentsPage`, `ModelsPage`, `ArtifactsPage`.

2.2.2 useModal

Composable управляет состоянием модального окна. До рефакторинга каждая страница объявляла отдельный флаг `showCreate: false` и методы его переключения:

```
1 import { ref } from 'vue'
2
3 export function useModal() {
4   const isOpen = ref(false)
5
6   function open()   { isOpen.value = true }
7   function close()  { isOpen.value = false }
8   function toggle() { isOpen.value = !isOpen.value }
9
10  return { isOpen, open, close, toggle }
```

11 }

Листинг 3: Файл src/composables/useModal.js

Composable применяется в: ExperimentsPage, ModelsPage, ArtifactsPage.

2.2.3 useListFilter

Composable реализует фильтрацию реактивного списка по строке поиска и дополнительному полю с применением фиксированных (применяемых по кнопке) значений фильтра:

```
1 import { ref, computed } from 'vue'
2
3 export function useListFilter(list, matchFn) {
4   const search      = ref('')
5   const extraFilter  = ref('')
6   const activeSearch = ref('')
7   const activeExtra  = ref('')
8
9   const filtered = computed(() =>
10     list.value.filter(
11       item => matchFn(item, activeSearch.value, activeExtra.value)
12     )
13   )
14
15   function apply() {
16     activeSearch.value = search.value
17     activeExtra.value  = extraFilter.value
18   }
19
20   function reset() {
21     search.value = extraFilter.value = ''
22     activeSearch.value = activeExtra.value = ''
23   }
24
25   return { search, extraFilter, filtered, apply, reset }
26 }
```

Листинг 4: Файл src/composables/useListFilter.js

Composable применяется в ExperimentsPage: фильтрация по имени и статусу эксперимента.

Переход на Composition API (<script setup>)

Все представления и компоненты переписаны с Options API на <script setup> Composition API. В качестве примера — страница экспериментов до и после рефакторинга.

До рефакторинга (Options API):

```
1 export default {
2   data() {
3     return {
4       showCreate: false,
5       creating:   false,
```

```

6      createError:'',
7      newExp: { name: '', tags: '', status: 'active' }
8    }
9  },
10  computed: {
11    ...mapState(useExperimentsStore, ['experiments'])
12  },
13  methods: {
14    async doCreate() {
15      this.creating = true
16      this.createError = ''
17      try {
18        await this.createExperiment({ ...this.newExp, ... })
19        this.showCreate = false
20      } catch (e) {
21        this.createError = e.message
22      } finally {
23        this.creating = false
24      }
25    }
26  },
27  mounted() { this.loadExperiments() }
28 }

```

Листинг 5: Фрагмент ExperimentsPage.vue — Options API (было)

После рефакторинга (<script setup> + composables):

```

1  const modal          = useModal()
2  const createAction = useAsyncAction()
3
4  const { search, extraFilter, filtered, apply, reset } =
5    useListFilter(experiments, (exp, s, status) => {
6      const matchName = !s || exp.name.toLowerCase()
7                        .includes(s.toLowerCase())
8      const matchStatus = !status || exp.status === status
9      return matchName && matchStatus
10    })
11
12  async function doCreate() {
13    if (!newExp.name) {
14      createAction.error.value = ' ',
15      return
16    }
17    await createAction.execute(async () => {
18      await experimentsStore.createExperiment({ ...newExp, ... })
19      modal.close()
20    })
21  }
22
23  onMounted(() => experimentsStore.loadExperiments())

```

Листинг 6: Фрагмент ExperimentsPage.vue — Composition API (стало)

Рефакторинг компонентов

Компоненты `AppSidebar.vue` и `AppTopbar.vue` переведены на `<script setup>`. В `AppTopbar.vue` логика темы перенесена из `mounted/methods` в `onMounted` и обычную функцию:

```
1 const isDark = ref(true)
2
3 onMounted(() => {
4   const saved = localStorage.getItem('mlpipe_theme')
5   const prefersDark =
6     !window.matchMedia('(prefers-color-scheme: light)').matches
7   isDark.value = saved ? saved === 'dark' : prefersDark
8   document.documentElement
9     .setAttribute('data-theme', isDark.value ? 'dark' : 'light')
10 })
11
12 function toggleTheme() {
13   isDark.value = !isDark.value
14   const theme = isDark.value ? 'dark' : 'light'
15   localStorage.setItem('mlpipe_theme', theme)
16   document.documentElement.setAttribute('data-theme', theme)
17 }
```

Листинг 7: `AppTopbar.vue` — управление темой (Composition API)

Лейаут `AppShell.vue` переписан с использованием `defineProps` и именованных слотов:

```
1 <template>
2   <div class="app-shell">
3     <SvgSprite />
4     <AppSidebar />
5     <div class="main-wrap">
6       <AppTopbar :title="title">
7         <template #actions>
8           <slot name="actions" />
9         </template>
10      </AppTopbar>
11      <div class="content-area">
12        <slot />
13      </div>
14    </div>
15  </div>
16 </template>
17
18 <script setup>
19 import SvgSprite from '@components/SvgSprite.vue'
20 import AppSidebar from '@components/AppSidebar.vue'
21 import AppTopbar from '@components/AppTopbar.vue'
22
23 defineProps({ title: { type: String, default: 'MLPipe' } })
24 </script>
```

Листинг 8: `AppShell.vue` — лейаут с именованным слотом

Роутер и navigation guard

Роутер подключён в `src/router/index.js` с ленивой загрузкой всех представлений и navigation guard, который защищает приватные маршруты:

```
1 router.beforeEach((to) => {
2   const auth = useAuthStore()
3   if (to.meta.requiresAuth && !auth.isLoggedIn)
4     return { path: '/login' }
5   if ((to.path === '/login' ||
6       to.path === '/register') && auth.isLoggedIn)
7     return { path: '/dashboard' }
8 })
```

Листинг 9: Navigation guard в `router/index.js`

Работа с внешним API через axios

API-слой организован через axios-инстанс с интерцептором авторизации. Каждый ресурс реализован отдельным классом, принимающим инстанс через конструктор (Dependency Injection):

```
1 class ExperimentsApi {
2   constructor(instance) { this.API = instance }
3
4   getAll = async () => this.API({ url: '/experiments' })
5   create = async (data) =>
6     this.API({ method: 'POST', url: '/experiments', data })
7   update = async (id, data) =>
8     this.API({ method: 'PATCH', url: `/experiments/${id}`, data })
9 }
```

Листинг 10: Фрагмент `src/api/experiments.js`

Все API-классы собраны в IoC-контейнере `src/api/index.js` и используются исключительно через Pinia-хранилища, что обеспечивает единую точку доступа к данным.

Запуск проекта

Проект запускается в двух терминалах:

```
1 # 1 --- (json-server)
2 node Server.js # http://localhost:3000
3
4 # 2 --- mlpipe-vue
5 npm install
6 npm run dev # http://localhost:5173
```

Листинг 11: Запуск бэкенда и фронтенда

Тестовые учётные записи: `admin@ml.pipe` / `admin123` и `user@ml.pipe` / `user123`.

Вывод

В ходе домашней работы приложение MLPipe мигрировано на Vue.js с выполнением всех поставленных требований.

Реализован полноценный слой composables (`useAsyncAction`, `useModal`, `useListFilter`), устранивший дублирование кода управления состоянием загрузки, ошибок, модальных окон и фильтрации списков. Все компоненты и представления переведены с Options API на `<script setup>` Composition API с использованием `ref`, `computed`, `onMounted` и `storeToRefs`. Работа с внешним API реализована через `axios` с интерцептором токена и классовым API-слоем по паттерну `Dependency Injection`. Клиентская навигация обеспечена `Vue Router` с ленивой загрузкой маршрутов и `navigation guard`. Компонентная архитектура разделена на три уровня: представления (`views`), переиспользуемые компоненты (`components`) и лейауты (`layouts`).